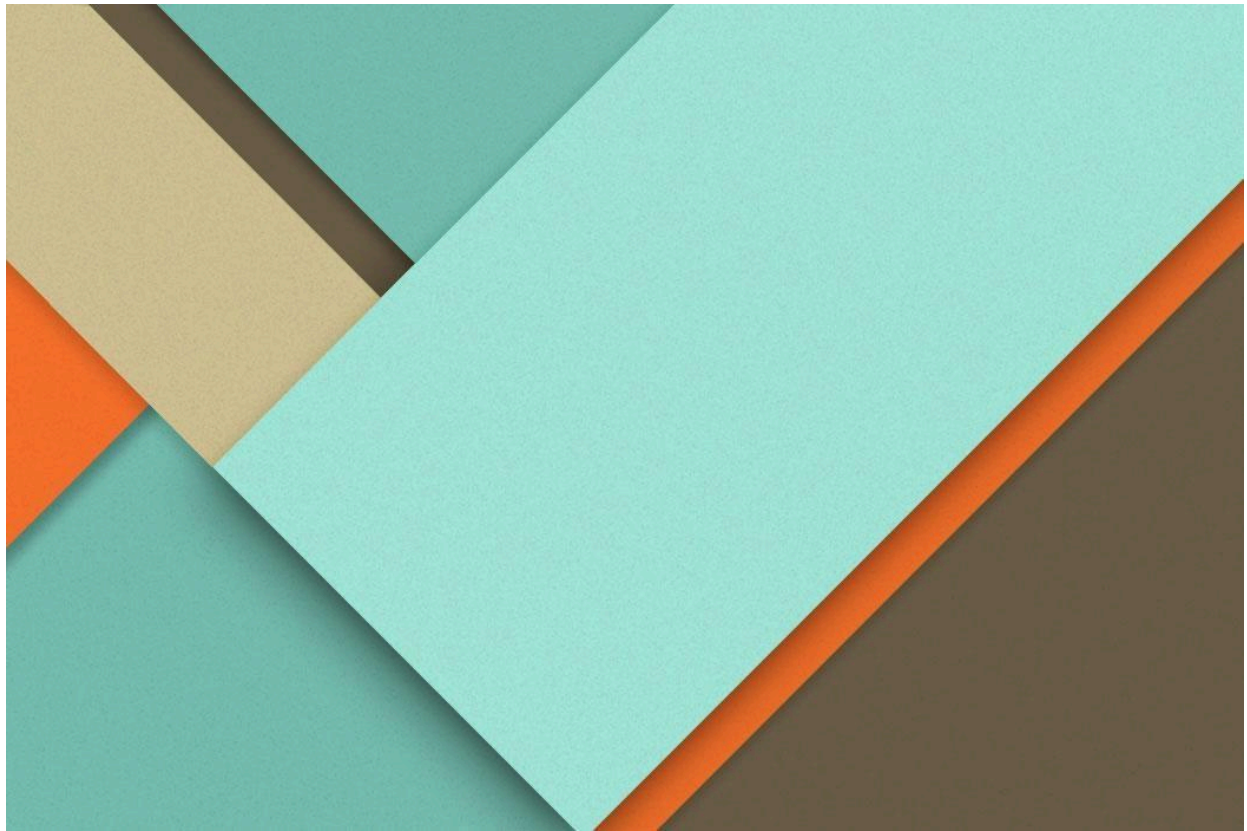




Modul Praktikum Pemrograman Berorientasi Obyek

Tahun 2024

Dwi Rolliawati

Program Studi Sistem Informasi - Fakultas Sains
dan Teknologi UIN Sunan Ampel Surabaya



MODUL 1 : Class & Method

Pendahuluan

Pada modul ini, mahasiswa akan mempelajari konsep dasar kelas dan method dalam Pemrograman Berorientasi Objek (PBO) menggunakan Java. **Kelas dan method** adalah elemen penting dalam PBO, di mana kelas berfungsi sebagai cetak biru (blueprint) untuk objek dan method digunakan untuk memberikan perilaku pada objek.

Tujuan

1. Memahami bagaimana mendefinisikan dan membuat sebuah kelas.
2. Menerapkan method pada kelas untuk memberikan perilaku pada objek.
3. Memahami konsep instansiasi kelas untuk membuat objek.

Praktikum 1 : Pengenalan Class & Method

1. Memahami cara mendefinisikan dan membuat kelas.
2. Membuat atribut dalam kelas untuk menyimpan informasi objek.
3. Memahami bagaimana mendefinisikan method dalam kelas.
4. Membuat method yang melakukan operasi pada atribut kelas.

Class

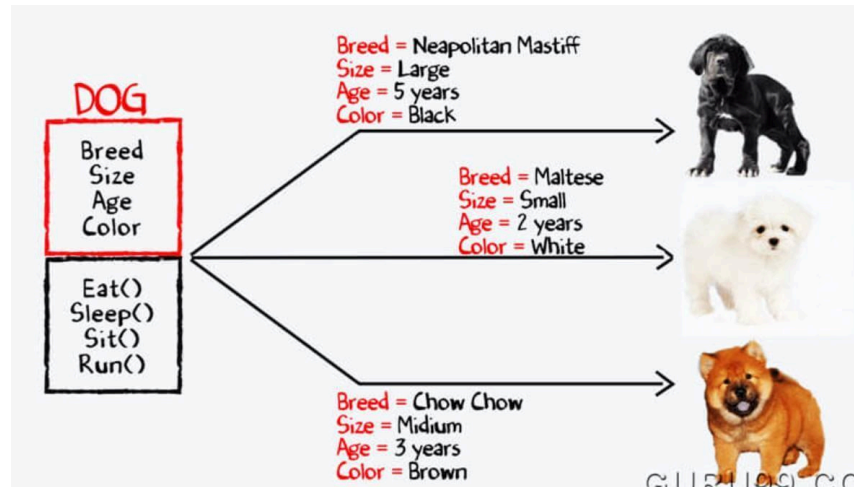
Sebuah kelas dapat berisi tipe-tipe variabel berikut:

Variabel lokal: Variabel yang didefinisikan di dalam metode, konstruktor, atau blok disebut variabel lokal. Variabel ini akan dideklarasikan dan diinisialisasi di dalam metode, dan variabel tersebut akan dihancurkan ketika metode telah selesai dieksekusi.

Variabel instans: Variabel instans adalah variabel yang berada di dalam kelas tetapi di luar metode apa pun. Variabel-variabel ini diinisialisasi saat kelas diinstansiasi. Variabel instans dapat diakses dari dalam metode, konstruktor, atau blok dari kelas tersebut.

Variabel kelas: Variabel kelas adalah variabel yang dideklarasikan di dalam kelas, di luar metode apa pun, dengan kata kunci static.

Mari kita ambil contoh dalam mengembangkan hewan peliharaan sistem manajemen, khusus ditujukan untuk anjing. Anda akan memerlukan berbagai informasi tentang anjing seperti ras anjing yang berbeda, umur, ukuran, dll. Kita perlu memodelkan makhluk nyata, misalnya anjing, menjadi entitas perangkat lunak sebagaimana gambar berikut ini.



Dari gambar diatas maka bisa di ilustrasikan kode programnya sebagai berikut :

```
// Class Declaration
public class Dog {
    // Instance Variables
    String breed;
    String size;
    int age;
    String color;

    // method 1
    public String getInfo() {
        return ("Breed is: "+breed+" Size is:"+size+" Age is:"+age+" color is: "+color);
    }

    public static void main(String[] args) {
        Dog maltese = new Dog();
        maltese.breed="Maltese";
        maltese.size="Small";
        maltese.age=2;
        maltese.color="white";
        System.out.println(maltese.getInfo());
    }
}
```

Saat membuat kelas, seseorang harus mengikuti berikut prinsip **SOLID** sebagai berikut:

1. Prinsip Tanggung Jawab Tunggal (**Single Responsibility Principle/SRP**)- Sebuah kelas seharusnya hanya memiliki satu alasan untuk berubah.

```
public class Task{
    public void downloadFile(String location);
    public void parseFile(File file);
    public void persistData(Data data);
}
```

2. Tanggung Jawab Terbuka Tertutup (**Open/Closed Principle** atau OCP)- Itu harus dapat memperluas kelas apa pun tanpa mengubahnya.

```
public double Area(Object[] shapes) {
    double area = 0;
    for (Object shape : shapes){
        if(shape is Rectangle){
            Rectangle rectangle = (Rectangle) shape;
            area+= rectangle.width*rectangle.height;
        }
    }
    return area;
}
```

3. Tanggung Jawab Pergantian Liskov (**Liskov Substitution Principle**/LSR)- Kelas turunan harus dapat menggantikan kelas dasarnya.

```
public abstract Shape{
    public abstract double area();
}

public class Rectangle extends Shape{
    public double width;
    public double height;
    //getters and setters
    public override double area(){
        return width*height;
    }
}

public class Circle extends Shape{
    public double radius;
    //getters and setters
    public override double area(){
        return radius*radius*Math.PI;
    }
}
```

4. Prinsip Pemisahan Antarmuka (**Interface Segregation Principle**/ISP)- Siapkan antarmuka terperinci yang spesifik untuk klien.
5. Prinsip Pembalikan Ketergantungan (**Dependency Inversion Principle**/DIP)- Bergantung pada abstraksi dan bukan pada konkresi

Method

Method adalah kumpulan program yang mempunyai nama. Program harus dibungkus dalam method. Dengan method kita bisa memanggil kumpulan program hanya dengan memanggil nama method-nya, pekerjaan jadi lebih singkat dan tidak boros menuliskan program, program menjadi lebih terstruktur, praktis, dan efisien

Ada 2 jenis method:

1. Method yang tidak mengembalikan data (diberi tipe **void**)
Contoh: static void cetak();
//method static bernama cetak yang tidak mengembalikan nilai.

2. Method yang bisa mengembalikan nilai menggunakan statement **return** dan tipe data

Contoh: static int cetak(int b);

return b;

//method bernama cetak yang mengembalikan nilai integer

//mempunyai 1 parameter/argument yaitu b dan bertipe integer

//return adalah keyword untuk mengembalikan nilai

Catatan:

- method yang dipanggil dari dalam method **static** harus **static**.
- method **tidak static** hanya bisa dipanggil melalui method **tidak static** dan mekanisme instansiasi.
- Jika memanggil method dalam satu kelas method harus static.
- method static hanya bisa memanggil method yang static

I. Percobaan 1

Silakan membuat kelas dengan kode program sebagai berikut

```
public class Puppy {
    int puppyAge;

    public Puppy(String name) {
        // This constructor has one parameter, <i>name</i>.
        System.out.println("Name chosen is : " + name );
    }

    public void setAge( int age ) {
        puppyAge = age;
    }

    public int getAge( ) {
        System.out.println("Puppy's age is : " + puppyAge );
        return puppyAge;
    }

    public static void main(String []args) {
        /* Object creation */
        Puppy myPuppy = new Puppy( "tommy" );

        /* Call class method to set puppy's age */
        myPuppy.setAge( 2 );

        /* Call another class method to get puppy's age */
        myPuppy.getAge( );

        /* You can access instance variable as follows as well */
        System.out.println("Variable Value : " + myPuppy.puppyAge );
    }
}
```

II. Percobaan 2

```
public class contoh{
    static void cetak(){
        System.out.println("Saya suka");
    }

    public void static (String [] args){
        cetak();
        System.out.println("Java");
    }
}
```

III. Percobaan 3

```
public class Main {
    // Static method
    static void myStaticMethod() {
        System.out.println("Static methods can be called without creating objects");
    }

    // Public method
    public void myPublicMethod() {
        System.out.println("Public methods must be called by creating objects");
    }

    // Main method
    public static void main(String[] args) {
        myStaticMethod(); // Call the static method
        // myPublicMethod(); This would output an error

        Main myObj = new Main(); // Create an object of Main
        myObj.myPublicMethod(); // Call the public method
    }
}
```

IV. Tugas

1. Apa yang dimaksud dengan kelas dalam PBO?
2. Bagaimana cara kerja atribut dan method dalam kelas?
3. Apa perbedaan antara method yang mengembalikan nilai dan yang tidak?
4. Bagaimana method berinteraksi dengan atribut dalam kelas?
5. Pada percobaan 1, definisikan bagian kode program sesuai prinsip SOLID.
6. Compile program pada percobaan 2 dan 3 dan lakukan analisa anda

Lembar Penyelesaian

1. Apa yang dimaksud dengan kelas dalam PBO?
Kelas adalah cetak biru (blueprint) yang mendefinisikan atribut (data) dan method (perilaku) untuk membentuk objek. Objek adalah instans dari kelas.
2. Bagaimana cara kerja atribut dan method dalam kelas?
Atribut menyimpan keadaan (state) objek; method mengoperasikan atau menampilkan state tersebut. Method dapat membaca/mengubah atribut instans (via **this**) atau atribut kelas (jika **static**)
3. Apa perbedaan antara method yang mengembalikan nilai dan yang tidak?
 - *Return value*: menghasilkan nilai (menggunakan **return**)
 - *Void*: menjalankan aksi tanpa mengembalikan nilai
4. Bagaimana method berinteraksi dengan atribut dalam kelas?
Method instans dapat membaca/memodifikasi atribut instans untuk mengubah state objek; method **static** hanya mengakses atribut **static** secara langsung.
5. Pada percobaan 1, definisikan bagian kode program sesuai prinsip SOLID.
 - **SRP**: Kelas Puppy hanya bertanggung jawab mengelola data terkait puppy (usianya, nama dll). Tidak mencampurkan logic lain.
 - **OCP**: Kita dapat memperluas kelas Puppy tanpa mengubah kelas dasar. Bisa diperluas (misal tambah **play()**, **eat()**), tanpa mengubah perilaku yang ada.
 - **LSP**: Objek dari kelas turunan harus bisa menggantikan objek kelas induknya tanpa mengubah konsistensi dan fungsi program. Jika kelak ada **Guidepuppy extends Puppy**, ia harus bisa menggantikan **Puppy** tanpa merusak program.
 - **ISP**: Kelas hanya mengimplementasikan interface yang dibutuhkan. Jika butuh antarmuka khusus (misal **Vocalizable** dgn **makeSound()**), buat interface terpisah agar klien tidak "dipaksa" method yang tak perlu.
 - **DIP**: Ketika bergantung pada abstraksi (interface) untuk perilaku (mis. suara), kode lebih fleksibel daripada bergantung pada kelas konkret.
6. Compile program pada percobaan 2 dan 3 dan lakukan analisa anda.
 - Percobaan 2
 - Kesalahan pada penggunaan keyword static sebagai nama method.
 - Fungsi cetak() dideklarasikan secara static sehingga bisa dipanggil tanpa harus membuat objek.
 - Metode yang benar seharusnya public static void main(String[] args) sebagai entry point program.
 - Percobaan 3
 - Metode myStaticMethod adalah static, sehingga bisa dipanggil langsung tanpa membuat objek.
 - Metode myPublicMethod bukan static, sehingga harus dipanggil melalui objek.
 - Di dalam main(), mencoba memanggil myPublicMethod() secara langsung (tanpa objek) akan menyebabkan error kompilasi.
 - Pembuatan objek Main myObj = new Main(); memungkinkan memanggil myPublicMethod() dengan myObj.myPublicMethod().
 - Program ini mendemonstrasikan perbedaan utama pemanggilan metode static dan



non-static.

- Kesimpulan
 - Percobaan 2: Program gagal kompilasi karena kesalahan sintaksis pada method static. Keyword static tidak bisa dipakai sebagai nama method. Perbaiki method main dengan deklarasi yang benar sesuai standar Java.
 - Percobaan 3: Program berjalan lancar dan berhasil menunjukkan implementasi method static dan non-static. Menjelaskan pentingnya pembuatan objek untuk method non-static.